

Datenbanken

Prof. Dr. Philipp Bruland

Technische Hochschule Ostwestfalen-Lippe
Fachbereich: Informatik und Automation

Sommersemester 2026

Lernziele

- Wie arbeitet der DBMS-interne Optimierer?
- Welche Einstellung zur Performance-Steigerung lassen sich vornehmen?
- Kennenlernen von Coding-Guidelines zur Datenbankerstellung
- Verbindung zwischen Datenbank und Programmierung verstehen
- JDBC API kennenlernen
- Sicherheit bei der Entwicklung

Übersicht

- 1 Optimierung
Datenbankoptimierung
Der Weg zu einer guten Datenbank
- 2 Anwendungsprogrammierung
Verbindung zwischen Programmen und Datenbanken
JDBC (Java Database Connectivity)

Warten...

... kann unerwünschte Folgen haben!

0,1 Sek.: wird als normal (unverzüglich) wahrgenommen

1,0 Sek.: ist die Grenze, welche noch nicht als Unterbrechung wahrgenommen wird

3,0 Sek.: bei den meisten die Grenze zum Abbruch

>10 Sek.: ist eine Unterbrechung und resultiert in Abwendung

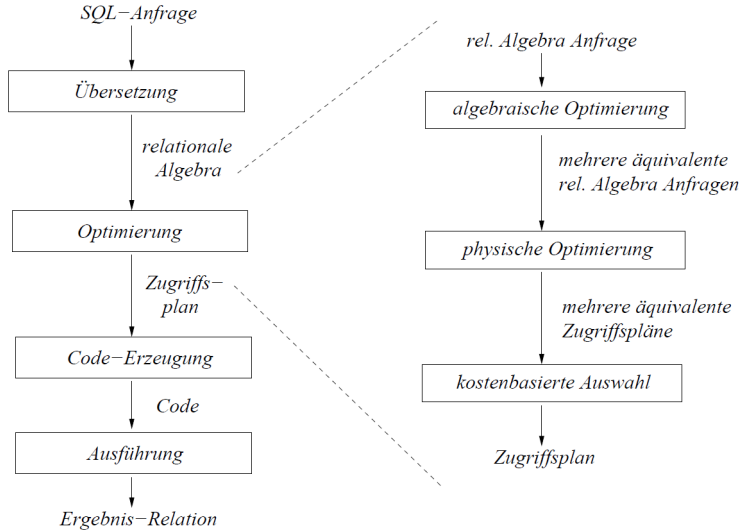
Quellen:

- Card SK, Moran TP, Newell A. The Psychology of Human-Computer Interaction. Erlbaum, Hillsdale, New Jersey, 1983.
- Newell A. Unified Theories of Cognition. Harvard University Press, Cambridge, Massachusetts, 1990.

Optimierung - Von Hause aus

- Das DBMS optimiert Anfragen selbst, um eine effiziente Ausführung zu ermöglichen
- DBMS Optimierer produziert manchmal suboptimale Ausführungspläne
- Die meisten DBMSe geben Benutzer*innen Einblick in die generierten Pläne
- Dieser kann analysiert und ggf. die Anfragen umgebaut werden.
- Manche DBMSe lassen Hinweise zur gezielten/optimierten Ausführung zu

Anfragebearbeitung durch ein DBMS



Anfrageauswertung allgemein

Vorbemerkung:

- Datenbanken können **sehr groß** werden
- werden auf Sekundärspeicher (Festplatte) gespeichert
- **Aufwand wird dominiert durch die Anzahl der Plattenzugriffe** („Seitenzugriffe“) Denn: In derselben Zeit, die für einen „Random Access“ auf der Festplatte benötigt wird, können zigtausende Operationen im Hauptspeicher durchgeführt werden.

Allgemeines Vorgehen:

- Erzeugen, Filtern, Manipulieren und Kombinieren von **Tupeln**
- Dabei evtl. Verwendung von Indexstrukturen („Wegweiser“), Hashing und Sortier-Schritten
- Wünschenswert: möglichst wenig auf Festplatte zwischenspeichern
- Operationen: Operationen der relationalen Algebra, Sortieren, Duplikateliminierung, ...

Algebraische Optimierung

- Ziel
 - Effiziente Ausführung eines algebraischen Ausdrucks
 - Minimierung der Größe von Zwischenergebnissen (das Endergebnis soll gleich bleiben!)
- Voraussetzung
 - Abschätzung der Größe von Zwischenergebnissen
- Verwendete Daten
 - Anzahl der Tupel in einer Relation R: $\text{Studenten}(R)$
 - Anzahl der unterschiedlichen Werte eines Attributes $A_i; j_i$
 - Vertauschungsregeln für Operationen
z. B.: $\sigma_{P_1}(\sigma_{P_2}(R)) = \sigma_{P_1 \wedge P_2}(R)$

Physische Optimierung

- Interne Auswahl von Algorithmen für Operationen
- Zugriff auf Tabellen
 - Scan, verschiedene Indizes, sortierter Zugriff, ...
- JOINS
 - Nested loop, sort-merge, hash, Joining (z. B.: Index-Join), ...
- Wähle für jede Operation die (hoffentlich) beste Implementierung

Kostenbasierte Auswahl

- Konzeptionell: Generiere alle denkbaren Anfrageausführungspläne
- Bewerte deren Kosten anhand eines Kostenmodells
 - Statistiken und Histogramme
 - Kalibrierung gemäß verwendeter Rechner
 - Abhängig vom verfügbaren Speicher
 - Aufwands-Kostenmodell
 - Führe „billigsten“ Plan aus

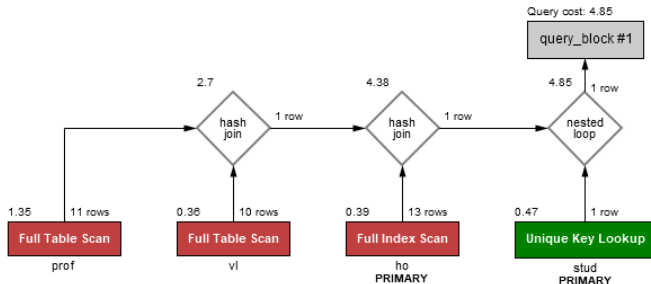
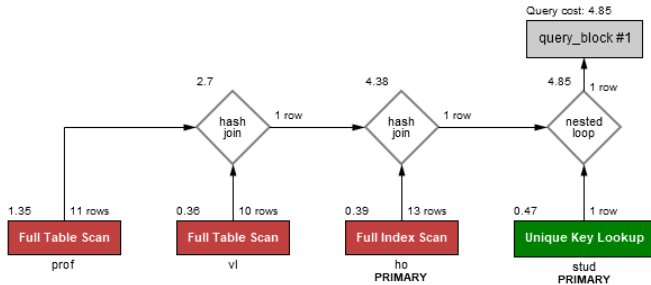
Anfrageoptimierung

Folgende Abfragen führen zu demselben Ergebnis. Sowohl vom Ausführungsplan als auch von der Zeit.

Aus Übung 09 . Aufgabe 25: WHERE und JOIN-Variante

```
1 SELECT stud.Name, stud.MatrNr
2 FROM studenten stud, hoeren ho, vorlesungen vl, professoren prof
3 WHERE stud.MatrNr = ho.MatrNr AND ho.VorlNr = vl.VorlNr
4 AND vl.gelesenVon = prof.PersNr AND prof.Name = 'Kant';
5
6 SELECT st.Name, st.MatrNr
7 FROM studenten st
8     INNER JOIN hoeren ho ON st.MatrNr = ho.MatrNr
9     INNER JOIN vorlesungen vl ON ho.VorlNr = vl.VorlNr
10    INNER JOIN professoren prof ON vl.gelesenVon = prof.PersNr
11 WHERE prof.Name = 'Kant';
```

Optimierter Ausführungsplan: WHERE und INNER JOIN



Welche Stellschrauben gibt es?

- Auf Datenbankebene
 - Passen die Datentypen zu meinen Attributen?

CHAR vs. VARCHAR

Wert	CHAR(4)	Benötigter Speicherplatz	VARCHAR(4)	Benötigter Speicherplatz
"	' '	4 Bytes	"	1 Byte
'ab'	'ab '	4 Bytes	'ab'	3 Bytes
'abcd'	'abcd'	4 Bytes	'abcd'	5 Bytes
'abcdefgh'	'abcd'	4 Bytes	'abcd'	5 Bytes

Integer-Typen

Typ	Speicher (Bytes)	Min. Wert	Max. Wert
TINYINT	1	-128	127
SMALLINT	2	-32768	32767
MEDIUMINT	3	-8388608	8388607
INT	4	-2147483648	2147483647
BIGINT	8	-2^{63}	$2^{63} - 1$

Welche Stellschrauben gibt es? (2)

- Auf Datenbankebene
 - Hat jede Tabelle angemessene Spalten?
 - Anwendungen, die sehr oft Updates vornehmen, haben meist *viele Tabellen mit wenigen Spalten*.
 - Anwendungen, die sehr große Datenmengen analysieren, haben meist *wenige Tabellen mit vielen Spalten*.
 - **innodb_buffer_pool_size:**
 - MySQL (sowie die meisten DBMSs auch) cachen Anfragen und deren Ergebnisse im **InnoDB buffer pool**.
 - Speicherzugriff ohne HDD-Zugriff
 - Typische Größenempfehlung: 50% - 70% des RAMs.

Welche Stellschrauben gibt es? (3)

- Auf Datenbankebene
 - Denormalisierung Erzeugen von Duplikaten, etc.
 - Komprimierung von Daten
 - Geringere Datenmengen werden zwischen Festplatte und Speicher transferiert
 - Benötigt weniger Festplatten- sowie RAM-Speicher
 - SSD vs. HDD

Kompression bei Tabellenerstellung

```
1 SET GLOBAL innodb_file_per_table=1;
2 SET GLOBAL innodb_file_format=Barracuda;
3 CREATE TABLE <table_name>
4 (<spalten_name> INT PRIMARY KEY)
5 ROW_FORMAT=COMPRESSED
6 KEY_BLOCK_SIZE=8;
```


Welche Stellschrauben gibt es? (4)

- Auf Datenbankebene
 - Optimizer Hints¹
 - DBMS-spezifische Hinweise (Kommentare) zur Optimierung von **RUDI** & **REPLACE**

Festlegen der Abarbeitungsreihenfolge von JOINS

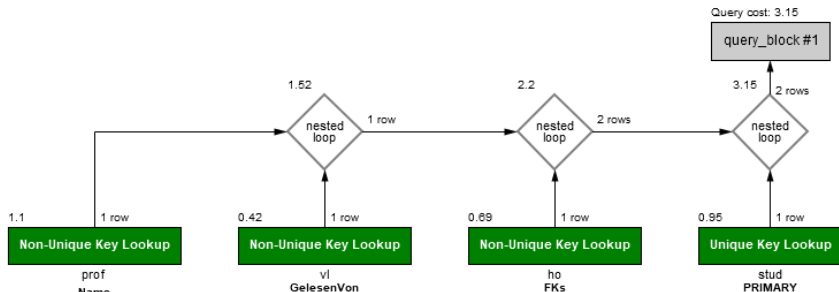
```
1 SELECT
2 /*+ JOIN_PREFIX(t2, t5@subq2, t4@subq1)
3     JOIN_ORDER(t4@subq1, t3) */
4 COUNT(*) FROM t1 JOIN t2 JOIN t3
5     WHERE t1.f1 IN (SELECT /*+ QB_NAME(subq1) */ f1
6                     FROM t4)
7     AND t2.f1 IN (SELECT /*+ QB_NAME(subq2) */ f1
8                   FROM t5);
```

- Bedachte Vergabe von **INDEX**-Spalten
 - Ziel: Anzahl der Tupel reduzieren, die bei einer Abfrage betrachtet werden.
 - In MySQL (InnoDB) mittels B-Trees realisiert
 - In der Regel für Spalten, die oft in WHERE-Klauseln verwendet werden
 - Zusammengesetzte **INDEXe** (z. B.: Vorname, Nachname, Geburtsdatum)

¹Weitere Infos in der MySQL-Spezifikation:

<https://dev.mysql.com/doc/refman/8.0/en/optimizer-hints.html#optimizer-hints-syn>

Welche Stellschrauben gibt es? - INDEX Vergabe



Bsp.: Nachträgliches anlegen eines INDEX für 'hören'

```
1 ALTER TABLE hochschuldatenbank.hoeren
2 DROP INDEX index_name ,
3 ADD INDEX index_name (MatrNr ASC, VorlNr ASC) VISIBLE;
```

Anfragekosten mit Index für abgefragte und Schlüssel-Spalten: 4,85 → 3,15

Welche Stellschrauben gibt es? (5)

- Auf Hardwareebene „KIWI-Prinzip: (Kill It With Iron)“
 - Zugriffszeiten einer Festplatte: Normal $< 10\text{ms}$ $\rightarrow$ 100 Suchen/Sekunde.
Optimierung: Mehrere (schnelle) Festplatten
 - CPU ...
 - RAM ...
- Bei produktiven Mehrbenutzersystemen wird das DBMS i. d. R. auf einen separaten Server ausgelagert bzw. auf mehrere Server verteilt (nicht Bestandteil der Vorlesung²).

²Umfassende Beschreibung der Thematik „Verteilte Datenbanken“: A. Kemper.
Datenbanksysteme - Eine Einführung. Oldenbourg Verlag München. S. 461-509

Übersicht

1 Optimierung

Datenbankoptimierung

Der Weg zu einer guten Datenbank

2 Anwendungsprogrammierung

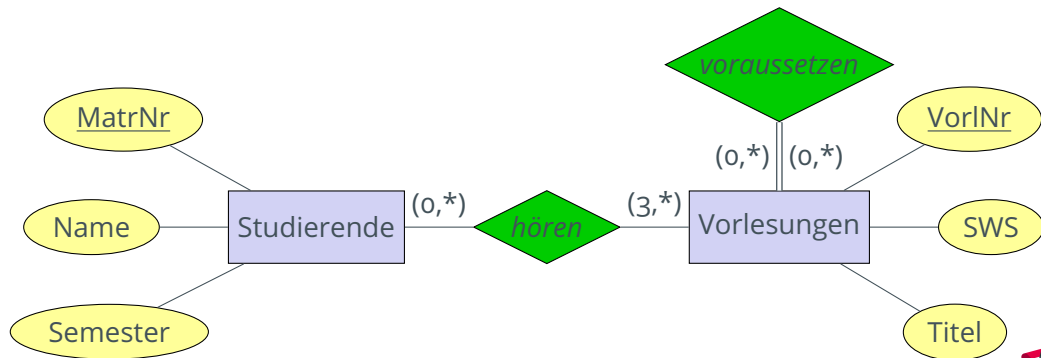
Verbindung zwischen Programmen und Datenbanken

JDBC (Java Database Connectivity)

Der Weg zu einer guten Datenbank

Dokumentation zur Nachvollziehbarkeit

- Konzeptuelles Modell
 - Entity-Relationship-Modell oder Klassenmodell nach UML
 - Minimale und maximale Kardinalitäten in beiden Richtungen
 - Wenn Rollen zu einer Beziehung, dann in beide Richtungen



Der Weg zu einer guten Datenbank

- Logisches Datenmodell als Relationenschema
 - Domänen sind als eigenständige Datenbankobjekte dokumentiert
 - Jedem Attribut ist die Domäne zugeordnet, die seine Wertemenge am genauesten beschreibt
 - Schlüssel sind dokumentiert (als PRIMARY KEY oder UNIQUE).
 - Fremdschlüssel sind angegeben. Dokumentation von Verhalten beim Ändern und Löschen (Integrität).
 - Doku von Konsistenzbedingungen, die nicht mit Schlüsseln, Fremdschlüsseln oder Domänen zusammenfallen
 - Ebenso Konsistenzbedingungen, die mehr als eine Relation betreffen

Der Weg zu einer guten Datenbank

Physikalisches Schema für die Datenbank

- Datentyp ggf. mit **CHECK**-Bedingung festlegen
- Datentyp eines Attributs orientiert sich an seiner Wertemenge
- Alle Schlüssel und Constraints realisieren
- Alle Konsistenzbedingungen (Constraints) haben Namen
- Nicht umsetzbare Konsistenzbedingungen (z. B. Assertions) werden in den Befehlen zur Datendefinition als Kommentare dokumentiert.

Normalformen

- Normalformen soweit wie möglich und sinnvoll umsetzen
- Denormalisierung durchführen wo es ebenfalls Sinn macht; bspw. aus Performance-Gründen.

Der Weg zu einer guten Datenbank

Namenskonventionen

- ① Namen für Relationen folgen einer einheitlichen Regel. Bezeichnung der darin zu speichernde Objektart durchgehend im Singular oder stets im Plural.
- ② Einsatz von Unterstrichen `_` und Sonderzeichen in Bezeichnern folgt einer einheitlichen Regel. Wird ein Attribut als *kunden_nr* bezeichnet, heißt es ebenfalls *artikel_nr* und nicht *artikelnummer* oder *artikelnr*.
- ③ Bezeichner für Attribute sollten möglichst aussagefähig sein bezüglich der inhaltlichen Bedeutung.
- ④ Gleichbedeutende Attribute in verschiedenen Relationen haben möglichst denselben Bezeichner.
- ⑤ Aussagekräftige Namen für Constraints. Die Art der Konsistenzbedingung kann durch Präfixe wie PK_, FK_, CH_ oder Ähnliches gekennzeichnet werden.

Weitere Gedanken...

- Mengengerüst der Datenbank
- Datenübernahme
- Datenschutzkonzept
- Datensicherungskonzept
- Verteilung
- Führung einer Historie

Übersicht

- 1 Optimierung
 - Datenbankoptimierung
 - Der Weg zu einer guten Datenbank
- 2 Anwendungsprogrammierung
 - Verbindung zwischen Programmen und Datenbanken
 - JDBC (Java Database Connectivity)

Wiederholung

3 Gruppen von SQL-Anweisungen:

DDL Data Definition Language

- Verwaltung von Metadaten
- Bsp.: **CREATE, DROP, ALTER**

DML Data Manipulation Language

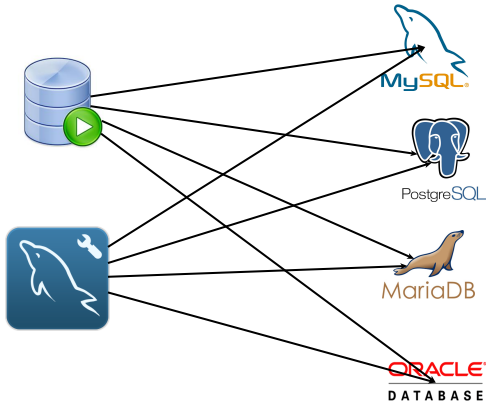
- RUDI (Read, Update, Delete, Insert):
- **SELECT, UPDATE, DELETE, INSERT**

DCL Data Control Language

- Verwaltung von Benutzern und Zugriffsrechten
- **CREATE USER, GRANT, REVOKE**
- Stored Procedure & Function
- Transaktionen
- Datenintegrität
- Trigger

Wo wollen wir hin?

- Bisher haben wir uns auf der Programmebene von Datenbankadministratoren bewegt
- SQL-Editoren sind gut für den Aufbau und die Verwaltung von Datenbanken



Wo wollen wir hin?

- Anwendungen mit unserer Datenbank verbinden:
 - Stand-alone Anwendungen
 - Mehrbenutzer-Betrieb
 - Web-Anwendungen
- Mehrere Entwicklungsschichten:
 - Software-Programmiersprache: C, Java, PHP, Python, ...
 - Datenbank-Programmiersprache: SQL, XQL, QBE, ...



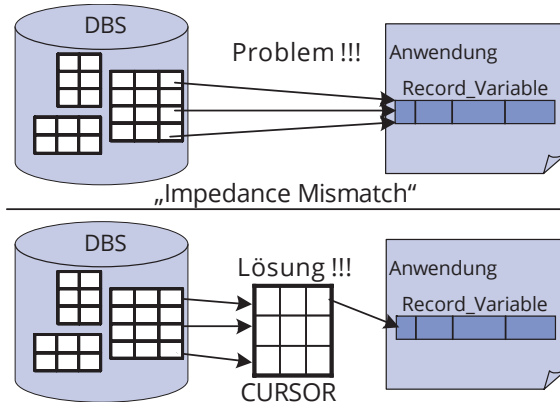
Übersicht

- 1 Optimierung
 - Datenbankoptimierung
 - Der Weg zu einer guten Datenbank
- 2 Anwendungsprogrammierung
 - Verbindung zwischen Programmen und Datenbanken
 - JDBC (Java Database Connectivity)

Verbindung zwischen Programmen und Datenbanken

- Client
 - Windows | Linux | MAC-Anwendung
 - Web-Anwendung (Applets / Servlets, ...)
- Server
 - Treiber
 - Datenbank
- Prinzip
 - Das Anwendungsprogramm 'sendet' SQL-Befehle an die Datenbank
 - Diese Datenbank liefert das Ergebnis (Ergebnismengen) zurück
 - Request/Reply-Architektur

Anwendung ↔ Datenbank - Impedance Mismatch



Anpassungsprobleme resultieren aus der Abbildung von Konzepten relationaler Datenbanken auf die Konzepte aus der Programmiersprachenwelt.

Gegenüberstellung: Impedance Mismatch

Relationales Datenbanksystem	Programmiersprache
Zentraler Datentyp für die Kollektionen von Daten: Tabelle	Zahlreiche Datentypen mit unterschiedlicher Semantik und unterschiedlichen Operatoren: Mengen, Listen, Bäume, Hash-Tabellen, ...
Mengenwertige Operationen - jede einzelne DML-Operation kann sehr viele Datensätze betreffen.	Einzelsatzoperationen - sollen mehrere Datensätze verändert werden, müssen Schleifen genutzt werden.
Attribute aller Datentypen können den Wert „unbekannt“ (NULL) annehmen	Attribute müssen immer einen Wert aus dem ihnen zugeordneten Wertebereich annehmen.
Dreiwertige Logik bei Aussagen (TRUE, FALSE, UNKNOWN)	Eine Aussage ist immer entweder TRUE oder FALSE.

Ansprechen der Datenbank aus einer Anwendung heraus

- Integration der Datenbankschnittstelle in eine Programmiersprache
- Unterschiedliche Ansätze:
 - **Embedded SQL** (wird nicht weiter vertieft)
 - Erweiterung einer konventionellen Programmiersprache um zusätzliche Sprachelemente.
 - Anweisungen werden in die Programmiersprache eingebettet.
 - Prä-Compiler übersetzt sie in API-Aufrufe.
 - Z. B.: ESQL/C, ESQL/Cobol, SQL/J, LINQ

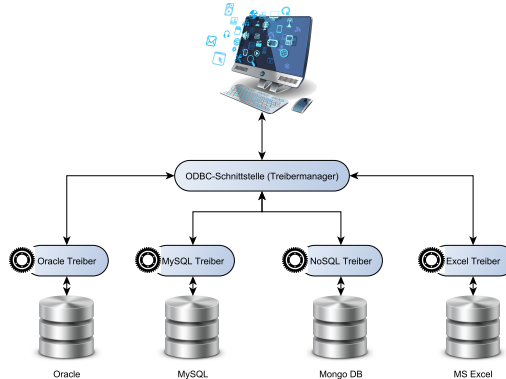
C-Programm mit Embedded SQL

```
1 int id = 42;
2 printf("Reihe mit id=42:\n");
3 EXEC SQL DECLARE c1 CURSOR FOR
4     SELECT name
5     FROM reihen
6     WHERE id = :id;
```

Ansprechen der Datenbank aus einer Anwendung heraus (2)

Weitere Ansätze: **ODBC** (Open Database Connectivity)

- Aufrufsschnittstelle von Microsoft implementiert
- Existiert auch für weitere Plattformen
- Verbindet den Datenbank-spezifischen Treiber mit einem Treibermanager
- Leitet die Anfragen und Ergebnisse direkt weiter ohne Neuüberstzung



Übersicht

- 1 Optimierung
 - Datenbankoptimierung
 - Der Weg zu einer guten Datenbank
- 2 Anwendungsprogrammierung
 - Verbindung zwischen Programmen und Datenbanken
 - JDBC (Java Database Connectivity)

JDBC - Das ODBC-Pendant zu ODBC

- JDBC (Java Database Connectivity) ist eine API, die zum Java Development Kit (JDK) gehört.
- Entwicklung von Datenbank- und Plattformunabhängigen Verbindungen
- RDBMS-spezifische Java Bibliothek erforderlich. Bspw. gibt es Treiber für:
 - C / C++
 - Java
 - .NET
 - Node.js
 - Python
 - und den ODBC...
- Soll das Programm mit einem anderen RDBMS kommunizieren, muss der Treiber ausgetauscht werden.
- MySQL Download (Connector/J): <https://dev.mysql.com/downloads/>

Eigenschaften von JDBC

- Dynamisches SQL
 - JDBC unterstützt ANSI SQL92-Basisfunktionalität
 - SQL-Befehle werden in Java 'nur' als Zeichenkette (String) behandelt - und nicht analysiert
 - Es steht erst zur Laufzeit fest, welcher SQL-Befehl ausgeführt wird. Man spricht daher auch von dynamischem SQL
 - Nachteil: Fehler in den SQL-Anweisungen werden nicht durch den Compiler, sondern erst zur Ausführungszeit erkannt
- Verbindungen zu Datenbanken
 - Es sind mehrere Verbindungen zu einer oder mehreren Datenbanken möglich

Prinzip der Anbindung

- Datenkonvertierung:
 - Abbildung der SQL-Datentypen auf die Java-Datentypen
- Cursor-Konzept
 - Abbildung der mengenorientierten Betrachtung in Datenbanken auf die elementweise Betrachtung in Java
- Prozeduraufrufe
 - Darstellung von SQL-Aufrufen über Java-Prozeduraufrufe

2 Programmier-Welten

- Paradigmenwechsel zwischen 'Datenbank-Sicht' und Programmiersprachen-Sicht:
- Datenbank-Sicht (Relationale Datenbanken, SQL, ...)
 - flache Typen
 - mengenorientiert
 - deklarativ
- Programmiersprachen-Sicht (Java, C++, ...)
 - strukturierte Typen
 - Objektorientierung
 - imperativ

SQL-Datentyp	Java-Datentyp
bigint	long
char(n)	java.lang.String
date	java.sql.Date
decimal(p,s)	java.math.BigDecimal
double	double
float	float
integer	int
smallint	short
time	java.sql.Time
timestamp	java.sql.Timestamp
varchar(n)	java.lang.String

Treibermanager

- In einem Java-Programm können gleichzeitig mehrere DBMS verwendet werden
- Die Verwaltung der geladenen Treiber wird durch den Treibermanager (Klasse: DriverManager) durchgeführt

JDBC: Wichtige Bestandteile

JDBC umfasst eine Menge von Klassen und Schnittstellen, die im Java-Package *java.sql* zusammengefasst sind. Die wichtigsten Klassen sind hierbei:

- *java.sql.DriverManager*: bildet den Einstiegspunkt, da hier die Treiber geladen und Verbindungen zur Datenbank aufgebaut werden können.
- *java.sql.Connection*: repräsentiert eine Datenbankverbindung
- *java.sql.Statement*: ermöglicht die Ausführung von SQL-Anweisungen über eine gegebene Verbindung
- *java.sql.ResultSet*: verwaltet die Ergebnisse einer Anfrage in Form einer Relation und unterstützt den Zugriff auf einzelne Spalten

Wir bleiben in Verbindung

Wie bei SQL-Dialogen, muss zunächst eine Verbindung zum RDBMS aufgebaut werden:

Connection

```
1 Connection conn = DriverManager.getConnection(  
2     "<API>:<DBMS>:<Art>//<HostURL>:<Port>/<Datenbank>",  
3     "<Benutzername>", "<Passwort>");
```

- **<API>**: Welche Art: ODBC oder JDBC (letzteres in unserem Fall)
- **<DBMS>**: Name der Datenbank (mysql)
- **<Art>**: Bei Oracle gibt es bspw. Thin-Treiber (Bei MySQL bleibt es leer)
- **<HostURL>**: Unter welcher Adresse ist der Server zu erreichen (auf demselben Server: localhost) Bei ORACLE wird anstatt von // ein @ verwendet.
- **<Port>**: Port-Adresse: Bei MySQL standardmäßig auf 3306
- **<Datenbank>**: Name der gewünschten Datenbank, auf die zugegriffen werden soll. (Kommt dem USE-Befehl aus SQL gleich)
- **Connection**-Objekt ist Ausgangspunkt für alle weiteren DB-Operationen

Wir bleiben in Verbindung

Hinweis:

Der Aufbau einer Verbindung zum RDBMS kann intensiv Ressourcen wie CPU-Zeit, Hauptspeicher und Netzwerkkapazitäten nutzen. Gehen Sie sparsam mit Datenbankverbindungen um.

Schließen von Verbindungen

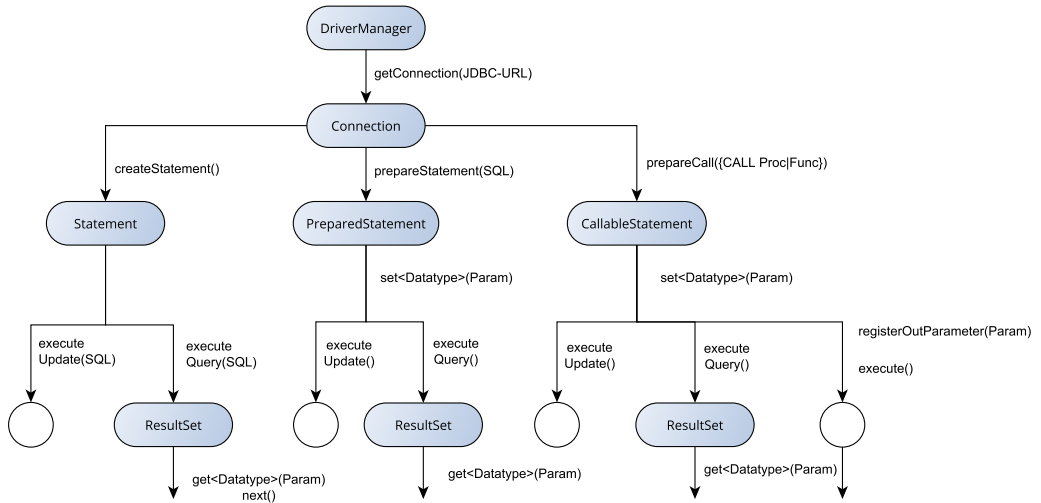
```
1 if (! conn.isClosed() )  
2     conn.close();
```

MySQL besitzt keinen Garbage-Collector, daher sollten die ungenutzten Verbindungen auch wieder geschlossen werden.

JDBC: Grundlegende Vorgehensweise

```
1 // Schritt 1: Verbindung aufbauen
2 // Laden des Treibers
3 Class.forName("com.mysql.cj.jdbc.Driver").newInstance();
4 Connection conn = DriverManager.getConnection("jdbc:mysql://
    localhost:3306/hochschuldatenbank", "user", "pw");
5
6 // Schritt 2: Erzeugen eines SQL-Statement-Objekts
7 Statement stmt = conn.createStatement();
8
9 // Schritt 3: Statement definieren; ResultSet entspricht CURSOR
10 String sql = "SELECT MatrNr, Name FROM studenten";
11 ResultSet rs = stmt.executeQuery( sql );
12
13 // Schritt 4: (Iterative) Abarbeitung der Ergebnisdatensätze
14 while (rs.next())
15     System.out.println ("MatrNr: " + rs.getString("MatrNr"));
16 rs.close();
17
18 // Schritt 5: Schließen der Datenbankverbindung
19 conn.close();
```

JDBC-Klassen - Aufrufbeziehungen (Ausschnitt)



Statement Ausführung

- Klasse *java.sql.Statement*
 - Ausführung von Anfragen (**SELECT**) mit **executeQuery**

```
1 String sql = "SELECT MatrNr, Name FROM studenten";  
2 ResultSet rs = stmt.executeQuery( sql );
```

- Ausführung von Änderungsanweisungen (**DELETE, INSERT, UPDATE**) mit **executeUpdate**

```
1 Statement stmt = conn.createStatement();  
2 String parameterMatrNr = 27550;  
3 String parameterVLNr = 5001;  
4 String sql = "UPDATE pruefen SET Note = 1.0 WHERE MatrNr =  
    "+parameterMatrNr+" AND VorlNr = "+parameterVLNr;  
5  
6 boolean success = stmt.executeUpdate( sql );
```

Ergebnisverarbeitung

- Klasse *java.sql.ResultSet*
 - Navigation über Ergebnismenge (Cursor-Prinzip)

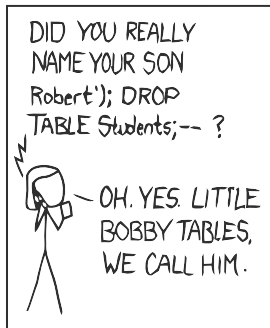
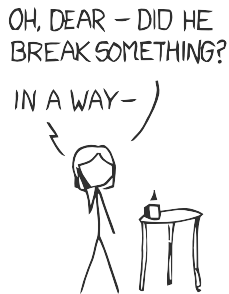
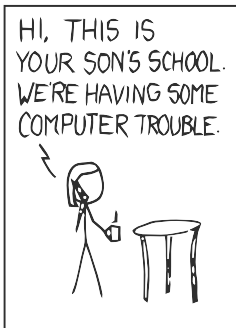
```
1 while (resultSet.next()) {  
2     // Verarbeitung der einzelnen Tupel  
3     ...  
4 }
```

- Zugriff auf Spaltenwerte über get<Datentyp>-Methoden
 - über Spaltenindex

```
1 String wName = resultSet.getString(1);
```

- über Spaltenname

```
1 String wName = resultSet.getString("Name");
```

Quelle: xkcd327

SQL Injections

Ungeprüftes Einschleusen von Benutzereingaben direkt in die SQL-Anweisung.

```
1 ...
2 // Unerwünschtes Selektieren
3 String benutzerEingabe = "2100 OR 1 = 1";
4 String sql = "SELECT * FROM professoren WHERE PersNr = " +
    benutzerEingabe;
5 stmt.execute(sql);
6 ...
7 // Oder ungewolltes Aktualisieren
8 String parameterMatrNr = '25232 OR Note > 0';
9 Statement stmt = conn.createStatement();
10 boolean success = stmt.executeUpdate("UPDATE pruefen SET Note =
    1.0 WHERE VorlNr = 5001 AND MatrNr = " + parameterMatrNr");
```

Schutz vor SQL-Injection-Attacken

- **PreparedStatement:** Vorkompilierte SQL-Anweisungen
 - Effizientere Methode zur Anweisungsausführung
 - SQL-Anweisung wird 1x an DBMS gesendet und dort vorübersetzt
 - Parameter werden anschließend nachgeladen und verarbeitet
 - Hier findet auch Prüfung von Parametern gegen unerlaubte Eingaben statt.

```
1 String sql = "SELECT Semester FROM studenten
2     WHERE Geburtstag > ? AND Name LIKE ? AND MatrNr > ? ";
3
4 PreparedStatement pstat = conn.prepareStatement(sql);
5 pstat.setObject(1, "1980-01-01"); // Parameterübergabe
6 pstat.setString(2, "Ar%");
7 pstat.setInt(3, 21000);
8
9 ResultSet rs = pstat.executeQuery();
10 while (rs.next()) {
11     System.out.println("Semester: " + rs.getString("Semester"));
12 }
```

Aufruf von Stored Programs

Geschieht mit der **CallableStatement** Routine.

Stored Functions & Procedures

```
1 CallableStatement cStmt;  
2 // Aufruf einer Prozedur  
3 cStmt = conn.prepareStatement("{CALL studyHighSemCount(?, ?)}");  
4 // Aufruf einer Funktion  
5 cStmt = conn.prepareStatement("{? = CALL studyHighSemCount(?, ?)}");  
6  
7 cStmt.setInt(1, 8);  
8 cStmt.registerOutParameter("studys", java.sql.Types.INTEGER);  
9  
10 cStmt.execute();  
11  
12 System.out.println("Anzahl: " + cStmt.getInt("studys") );
```

Transaktionskontrolle

- **Transaktionskontrolle** durch Methodenaufrufe der Klasse **Connection**
 - **setAutoCommit**: Ein-/Abschalten des Autocommit-Modus (jedes Statement ist eigene Transaktion)
 - **setReadOnly**: Festlegung ob lesende oder ändernde Transaktion
 - **setTransactionIsolation**: Festlegung der Synchronisationsanforderungen (None, Read Uncommitted, Read Committed, Repeatable Read, Serializable)
 - **commit** bzw. **rollback**: erfolgreiches Transaktionsende bzw. -abbruch

Beispiel für Transaktionskontrolle

```
1 try {
2     con.setAutoCommit (false);
3     // einige Änderungsbefehle, z. B. INSERTs, UPDATEs, etc.
4     con.commit ();
5 } catch (SQLException e) {
6     try { con.rollback (); } catch (SQLException e2) {}
7 } finally {
8     try { con.setAutoCommit (true); } catch (SQLException e3) {}
9 }
```

Weiterführende Literatur

- Baron Schwartz. High Performance MySQL. O'Reilly, 2008. ISBN: 978-0-596-10171-8

Vielen Dank!